

10 Different Examples of For Loops in Python

A Guide for BCA Students

Example 1: Basic For Loop with Range

Concept: Iterating through a sequence of numbers using `range()`

```
# Print numbers from 0 to 4
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

Explanation: The `range(5)` function generates numbers from 0 to 4. The loop variable `i` takes each value sequentially.

Example 2: For Loop with List

Concept: Iterating through elements in a list

```
fruits = ["Apple", "Banana", "Cherry", "Date"]
for fruit in fruits:
    print(f"I like {fruit}")
```

Output:

```
I like Apple
I like Banana
I like Cherry
I like Date
```

Explanation: The loop iterates through each element in the `fruits` list directly, without needing index values.

Example 3: For Loop with String

Concept: Iterating through each character in a string

```
name = "Python"
for char in name:
    print(char)
```

Output:

```
P
Y
t
h
o
n
```

Explanation: Strings are sequences in Python, so a for loop automatically iterates through each character.

Example 4: For Loop with Enumerate

Concept: Accessing both index and value simultaneously

```
colors = ["Red", "Green", "Blue", "Yellow"]
for index, color in enumerate(colors):
    print(f"Index {index}: {color}")
```

Output:

```
Index 0: Red
Index 1: Green
Index 2: Blue
Index 3: Yellow
```

Explanation: `enumerate()` returns both the index and the value, allowing you to track position while iterating.

Example 5: For Loop with Range (Start, Stop, Step)

Concept: Controlling the loop with custom start, stop, and step values

```
# Print even numbers from 2 to 10
for i in range(2, 11, 2):
    print(i)
```

Output:

```
2
4
6
8
10
```

Explanation: `range(2, 11, 2)` starts at 2, goes up to (but not including) 11, and increments by 2 each iteration.

Example 6: Nested For Loops

Concept: A loop inside another loop, useful for working with 2D data

```
# Create a multiplication table
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} × {j} = {i * j}", end=" ")
    print() # New line after inner loop completes
```

Output:

```
1 × 1 = 1   1 × 2 = 2   1 × 3 = 3
2 × 1 = 2   2 × 2 = 4   2 × 3 = 6
3 × 1 = 3   3 × 2 = 6   3 × 3 = 9
```

Explanation: The outer loop runs 3 times, and for each iteration, the inner loop runs 3 times, creating a 3×3 table.

Example 7: For Loop with Dictionary

Concept: Iterating through key-value pairs in a dictionary

```
student = {"name": "Raj", "age": 20, "course": "BCA"}

# Iterate through keys and values
for key, value in student.items():
    print(f"{key}: {value}")
```

Output:

```
name: Raj
age: 20
course: BCA
```

Explanation: The `.items()` method returns both keys and values, allowing you to process both in each iteration.

Example 8: For Loop with Break Statement

Concept: Exiting the loop early when a condition is met

```
# Find first even number in a list
numbers = [3, 7, 9, 12, 15, 20]
for num in numbers:
    if num % 2 == 0:
        print(f"First even number found: {num}")
        break
    print(f"Checking {num}...")
```

Output:

```
Checking 3...
Checking 7...
Checking 9...
First even number found: 12
```

Explanation: The `break` statement terminates the loop as soon as an even number is found, preventing further iterations.

Example 9: For Loop with Continue Statement

Concept: Skipping the current iteration and moving to the next one

```
# Print numbers, but skip multiples of 3
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i)
```

Output:

```
1
2
4
5
7
8
10
```

Explanation: When `i` is divisible by 3, the `continue` statement skips the `print()` and moves to the next iteration.

Example 10: For Loop with List Comprehension

Concept: A concise way to create new lists by iterating and transforming elements

```
# Square all numbers in a list
numbers = [1, 2, 3, 4, 5]
squared = [num ** 2 for num in numbers]
print(squared)

# Filter and transform: double all even numbers
doubled_evens = [num * 2 for num in numbers if num % 2 == 0]
print(doubled_evens)
```

Output:

```
[1, 4, 9, 16, 25]
[4, 8]
```

Explanation: List comprehension is a powerful Python feature that replaces verbose for loops with a single line of code. It's equivalent to writing a for loop to build a list.

Summary Table

Example	Key Concept	Use Case
1	Basic range()	Simple counting loops
2	List iteration	Processing list elements
3	String iteration	Character-by-character processing
4	Enumerate	When you need both index and value
5	Range with step	Custom sequences (even numbers, etc.)
6	Nested loops	Multi-dimensional data processing
7	Dictionary iteration	Working with key-value pairs
8	Break statement	Early loop termination
9	Continue statement	Skipping iterations
10	List comprehension	Concise list creation and filtering

Practice Tips for BCA Students

- Experiment with each example** in an IDE (VSCode, OnlineGDB, or Python IDLE)
- Modify the code:** Change ranges, list values, and conditions to see different outputs
- Combine concepts:** Try using `enumerate()` with `break` or `continue`
- Performance consideration:** For large datasets, list comprehension (Example 10) is often faster than traditional loops
- Real-world application:** Use nested loops for processing 2D arrays/matrices, which is common in data science

Common Mistakes to Avoid

- ❑ **Forgetting the colon (:)** at the end of the for statement

```
for i in range(5) # Wrong - missing colon
    print(i)
```

✓ **Correct syntax:**

```
for i in range(5): # Correct
    print(i)
```

- ❑ **Inconsistent indentation** - Python is very strict about this

```
for i in range(3):
print(i) # Wrong - not indented
```

✓ **Correct indentation:**

```
for i in range(3):
    print(i) # Correct - indented
```
